CHAPTER 10

Cache Memory Mapping

Direct · fully associative · set-associative · tag/index/offset · cache size in bits · replacement · write policies · miss types

QUICK TRICK

Address = **Tag | Index | Offset**. Offset = $\log_2$(block size). Direct: index = $\log_2$(lines). k-way: index = $\log_2$(lines / k). Fully associative: **no index**. Tag = address bits − index − offset. Cache bits = lines × (data + tag + valid [+ dirty]).

10.1 The problem and the idea

● **LEVEL 1 · BEGINNER**

The problem. The cache is much smaller than main memory. Which cache line may hold which memory block, and how do we find it? **The idea.** A **mapping function** decides. **Analogy.** A coat check: direct = your number fixes your hook; fully associative = any free hook; set-associative = any free hook in your row.

10.2 Three mappings

● **LEVEL 2 · EXAM**

Mapping	Where a block may go	Index	Comparators	Conflict misses
Direct-mapped	Exactly one line: (block number) mod (lines)	$\log_2$(lines)	1	Most
Fully associative	Any line	none	= number of lines	None
k-way set-associative	Any of k lines in set (block mod sets)	$\log_2$(lines / k)	k	In between

Direct-mapped is k = 1; fully associative is k = number of lines.

Worked example

32-bit address, 64 KB cache, 32 B blocks. Offset = 5 bits. Lines = $2^{16}/2^5 = 2^{11} = 2048$.

Mapping	Sets	Index	Tag
Direct-mapped	2048	11	32 − 11 − 5 = **16**
4-way	512	9	32 − 9 − 5 = **18**
Fully associative	1	0	32 − 5 = **27**

Total cache size (direct-mapped). Each line = 256 data bits + 16 tag + 1 valid = 273 bits. Total = 2048 × 273 = **559,104 bits ≈ 68.25 KB**. With a dirty bit (write-back): 2048 × 274 = 561,152 bits.

10.3 Tracing hits and misses

● **LEVEL 3 · GATE**

4 cache lines, 1-word blocks. Block addresses: **0, 4, 0, 8, 4, 12**. Compare the three mappings (LRU for associative).

Mapping	Where all of them go	Misses	Hits
Direct (4 lines)	all map to line 0 (address mod 4 = 0)	**6**	0
2-way (2 sets, set 0)	all in set 0 (mod 2 = 0); LRU evicts	**5**	1 (the second 0)
Fully associative (4 lines)	any line; only 4 distinct blocks	**4**	2 (0 and 4)

QUICK TRICK

More associativity → fewer conflict misses, but more comparators, more hardware and a longer hit time.

10.4 Replacement and write policies

● LEVEL 2 · EXAM

- **Replacement** (needed for associative caches): **LRU** (least recently used), **FIFO**, **random**. Direct-mapped needs none.
- **Write-through:** write to cache and memory together (simple, slow; use a write buffer). **Write-back:** write only the cache and set a **dirty bit**; write the line to memory when it is evicted.
- **Write miss: write-allocate** (load the block, then write; usual with write-back) or **no-write-allocate** (write straight to memory; usual with write-through).

10.5 Miss types and performance

● LEVEL 2 · EXAM

Miss	Cause	Reduced by
Compulsory (cold)	First access to a block	Larger blocks, prefetch
Capacity	Cache too small for the working set	Larger cache
Conflict	Too many blocks map to one set	More associativity

```
CPI = CPI_base + (memory accesses per instruction) x miss rate x miss penalty (cycles)
Example: 1.2 + 1.3 x 0.04 x 50 = 1.2 + 2.6 = 3.8
```

Block size effect. Larger blocks use spatial locality (fewer compulsory misses) but raise the miss penalty and, for a fixed cache, give fewer lines (more conflict misses).

10.6 Common mistakes

COMMON MISTAKE

- Taking the index as $\log_2$(lines) for a set-associative cache (use sets).
- Forgetting the valid bit (and dirty bit) when asked for the **total** cache size.
- Counting the tag from the wrong address width (use the physical address if the cache is physically addressed).
- Forgetting that fully associative has no index field.
- Using the byte address where the block address is needed in a hit/miss trace.

GATE QUICK CHECK — Chapter 10

```
offset = log2(block)   lines = cache/block   sets = lines/k   index = log2(sets)
tag = A - index - offset      direct: largest index, smallest tag | fully assoc: no index, largest tag
cache bits = lines x (8*block + tag + valid [+ dirty])      comparators = k
write-back: dirty bit, write on evict | write-through: always write memory
```

Practice questions

● LEVEL 1 · BEGINNER

Q1. Number of comparators in an 8-way set-associative cache.

Q2. Which miss type does full associativity eliminate?

Q3. Does a direct-mapped cache need a replacement policy?

Q4. What does a dirty bit mean?

● LEVEL 2 · EXAM

Q5. 32-bit address, 16 KB direct-mapped cache, 16 B blocks. Offset, index, tag bits?

Q6. Same cache but 4-way set-associative. Index and tag bits?

Q7. 64 KB direct-mapped cache, 32 B blocks, 32-bit address, valid bit only. Total bits?

Q8. Base CPI 1.2, 1.3 memory accesses per instruction, miss rate 4%, miss penalty 50 cycles. CPI?

● LEVEL 3 · GATE

Q9. Physical address 40 bits, 32 KB 4-way cache, 64 B blocks. Offset, index, tag?

Q10. Main memory 16 MB (24-bit address), 128 KB direct-mapped cache, 64 B blocks. Tag, index, offset?

Q11. Block addresses 0, 4, 0, 8, 4, 12 on a 4-line direct-mapped cache. Misses?

Q12. A 2-way cache with 2 sets, LRU, same sequence 0, 4, 0, 8, 4, 12. Misses?

Answers

Q	Solution
1	**8**.
2	**Conflict** misses.
3	**No**: each block has only one possible line.
4	The cache line has been modified and must be written back to memory on eviction (write-back cache).
5	Offset 4; lines 1024 → index 10; tag = 32 − 14 = **18**.
6	Sets = 256 → index **8**; tag = 32 − 8 − 4 = **20**.
7	2048 × (256 + 16 + 1) = **559,104 bits** (68.25 KB).
8	1.2 + 1.3 × 0.04 × 50 = **3.8**.
9	Offset 6. Lines = 512; sets = 128 → index **7**. Tag = 40 − 7 − 6 = **27**.
10	Offset 6; lines = 2048 → index 11; tag = 24 − 17 = **7**.
11	All map to line 0 → **6 misses**.
12	**5 misses**, 1 hit.

Next: Chapter 11 — Disk and Secondary Storage.

AMJESH singhamjesh.github.io